

Reporte de Sprint 2: Pipeline Reproducible de Ingeniería de Variables y Clasificación de Clientes Premium

Marcelo de la Quintana, Gaston Nina, y Gerick Toro
Módulo de Implementación de Soluciones de Inteligencia Artificial
Maestría en Inteligencia Artificial y Ciencia de Datos

Resumen—Este informe documenta el Sprint 2 del proyecto analítico sobre la plataforma Olist, cuyo objetivo central fue transformar el trabajo exploratorio del Sprint 1 en un pipeline modular y reproducible para la clasificación supervisada de clientes Premium (*is_premium*). Se migró la lógica del notebook histórico hacia cuatro scripts independientes en *src/*, se construyeron 18 nuevas variables operativas, logísticas, de cuotas y categoría, y se ejecutaron 8 experimentos de selección de variables con control explícito de fuga de información (*data leakage*). El experimento ganador (*corr_le_0.85*) seleccionó 28 features mediante filtro de correlación, con una diferencia en AUC-ROC de validación de apenas 0.0018 respecto al conjunto inicial de 37 features, a cambio de menor redundancia y mayor parsimonia. La evaluación final sobre un split temporal muestra mejoras sustanciales frente al baseline del Sprint 1 en todas las métricas clave: ROC-AUC pasa de 0.5355 a 0.7929, el F1-score de 0.21 a 0.54 y el Gini de 0.071 a 0.586. Los KPIs confirman que el segmento premium (20 % de 96.096 clientes) concentra el 56.03 % de la facturación observada, con un ticket promedio 4.36 veces superior al del cliente regular. Una simulación sobre el periodo de holdout (ago–oct 2018, 6.468 clientes activos) valida la estabilidad del umbral fijo: tasa premium de 19.37 % y ratio de ticket de 4.47x, coherentes con los valores del conjunto *dev*.

Index Terms—Pipeline Reproducible, Ingeniería de Variables, Selección de Variables, Clientes Premium, Olist, Fuga de Datos, Random Forest, ROC-AUC.

I. INTRODUCCIÓN

EL Sprint 1 estableció la viabilidad inicial del proyecto mediante la exploración de los datos crudos de Olist, la definición del target *is_premium* como etiqueta binaria basada en el percentil 80 del gasto acumulado de pedidos entregados, y la evaluación de un modelo baseline de regresión logística que arrojó un ROC-AUC de 0.5355. Este resultado evidenció la **Paradoja del Target**: el 97 % de los clientes posee una sola orden, por lo que las variables RFMT tradicionales de frecuencia y lifetime son prácticamente idénticas para el segmento premium y el regular.

El objetivo del **Sprint 2** fue resolver esa paradoja construyendo un conjunto enriquecido de variables de comportamiento, logística, método de pago y categoría de producto, y organizarlas en un pipeline modular que sea auditable, versionado y reproducible sin depender de la ejecución manual de notebooks.

I-A. Principio Rector del Sprint

Los notebooks históricos del Sprint 1 se mantienen como evidencia sin modificación. El Sprint 2 construye un flujo

separado en *src/* con tres capas conceptuales:

1. **Evidencia histórica**: notebooks de Sprint 1 intactos.
2. **Pipeline reproducible**: scripts en *src/data/* y *src/features/*.
3. **Exposición analítica**: notebook integrador que consume artefactos, sin regenerarlos.

II. ARQUITECTURA DEL PIPELINE Y DAG

II-A. Estructura General

El pipeline del Sprint 2 se organiza en cuatro capas funcionales con dependencias explícitas y artefactos intermedios persistidos en *data/interim/* y *data/processed/*. El orden de ejecución reproducible es:

1. *src/data/build_master_table.py*
2. *src/features/build_rfm_features.py*
3. *src/features/feature_selection_experiments.py*
4. *src/models/evaluate_model.py*

II-B. Entradas y Salidas por Etapa

La Tabla I resume los scripts, entradas y salidas principales de cada capa.

Cuadro I
RESUMEN DE ETAPAS DEL DAG — SPRINT 2

Etapa	Script	Entrada	Salidas principales
Master table	<i>build_master_table.py</i>	{ <i>parfil</i> }/*.parquet	01_raw_{ <i>parfil</i> }.parquet, 03_clean_{ <i>parfil</i> }.parquet, threshold.json
Features	<i>build_rfm_features.py</i>	03_clean_{ <i>parfil</i> }.parquet	05_features_rfm.parquet
Selección	<i>feature_selection_experiments.py</i>	05_features_rfm.parquet	06_experiments.parquet, 06_selected.parquet
Evaluación	<i>evaluate_model.py</i>	06_selected.parquet	08_evaluation_metrics.json
Análisis	<i>02_pipeline_features.ipynb</i>	Todos los artefactos	Gráficos, narrativa

II-C. Decisiones Clave del DAG

Cuatro decisiones de diseño son relevantes para la trazabilidad y defensa del pipeline:

- El script *select_features.py* fue deprecado. La fuente oficial de *06_features_selected.parquet* es *feature_selection_experiments.py*, para garantizar que la selección final esté respaldada por experimentación con métricas y no por criterio manual.
- El corte temporal oficial para selección y evaluación es **2018-07-01**. Se descartó *2018-08-01* porque la fecha máxima de *last_purchase* en el split *dev* es 2018-07-31, lo que dejaba el conjunto de validación vacío.

- El umbral del target se calcula una sola vez sobre la población *dev* (**P80 = BRL 197.01**) y se persiste en modo *fit*. Las corridas posteriores usan el modo *apply* sin recalcular, garantizando comparabilidad entre experimentos.
- Los artefactos de la master table usan un prefijo de perfil en el nombre (*_dev*, *_holdout*, *_raw*) para evitar colisiones entre ejecuciones sobre distintos conjuntos de datos. El script orquestador `scripts/build_features.sh` corre el pipeline completo: pasos 1–4 sobre el *split dev* y pasos 5–6 sobre el *holdout* con umbral en modo *apply*.

III. MASTER TABLE Y DEFINICIÓN DEL TARGET

III-A. Construcción de la Master Table

La lógica del notebook `01_build_master_table.ipynb` del Sprint 1 fue migrada al script `src/data/build_master_table.py`. Las reglas principales son:

- Granularidad final por `customer_unique_id`.
- Métricas financieras basadas exclusivamente en pedidos con estado `delivered`.
- Métricas operativas calculadas sobre todas las órdenes disponibles del cliente.
- Imputación mínima a 0 para conteos y sumas clave.

La master table raw (96.096×32 columnas) expuso valores faltantes en variables financieras y de comportamiento. Las columnas con mayor porcentaje de nulos se muestran en la Tabla II.

Cuadro II
VARIABLES CON MAYOR PORCENTAJE DE NULOS — MASTER TABLE RAW

Variable	Nulos	% Nulos
<code>avg_delivery_days</code>	8.884	9.24 %
<code>total_spent / avg_ticket</code>	8.882	9.24 %
<code>top_category</code>	8.205	8.54 %
<code>avg_review_score</code>	6.966	7.25 %
<code>max_item_price</code>	6.914	7.19 %
<code>payment_methods_count</code>	6.278	6.53 %

Tras la limpieza (Fase 4), la master table limpia quedó con **96.096 filas × 33 columnas** y **cero duplicados** por `customer_unique_id`.

III-B. Definición y Defensa del Target

El target *is_premium* se mantiene como etiqueta binaria sobre gasto acumulado de pedidos entregados, con umbral fijo:

$$is_premium = \begin{cases} 1 & \text{si } total_spent \geq P80_{dev} = 197,01 \\ 0 & \text{en caso contrario} \end{cases} \quad (1)$$

Esta definición produce una tasa premium del **20.00 %** (19.220 clientes) sobre 96.096 clientes totales. Las justificaciones metodológicas son:

- **Por qué P80:** conserva un segmento interpretable del ~20 %, consistente con la lógica de segmentación de alto

valor; ni demasiado amplio (P75) ni demasiado estrecho (P90). Los KPIs de negocio validan la elección: ese 20 % explica el 56.03 % de la facturación.

- **Por qué umbral fijo:** evita que el target cambie en cada corrida del pipeline, manteniendo comparabilidad entre experimentos y corridas futuras con nueva data.
- **Por qué excluir cancelaciones:** el target representa valor efectivamente realizado, no intención de compra. Incluir pedidos no entregados mezclaría ingreso concretado con operaciones fallidas.

IV. INGENIERÍA DE VARIABLES

IV-A. Nuevas Variables Creadas

El módulo `src/features/build_rfm_features.py` generó **18 variables nuevas** sobre la master table limpia, agrupadas por dimensión en la Tabla III.

Cuadro III
VARIABLES NUEVAS CREADAS EN EL SPRINT 2

Dimensión	Variables
Composición de pedido	<code>items_per_order</code> , <code>products_per_order</code> , <code>max_to_avg_price_ratio</code>
Flete	<code>freight_to_item_ratio</code>
Cuotas	<code>installments_gt_1_flag</code> , <code>installments_gt_6_flag</code>
Método de pago	<code>credit_card_flag</code> , <code>boleto_flag</code> , <code>voucher_flag</code> , <code>payment_complexity_flag</code>
Logística	<code>has_late_delivery</code> , <code>has_cancellation</code> , <code>delivery_gap</code>
Reseñas	<code>reviews_per_order</code>
Geografía	<code>region_group</code> , <code>far_region_flag</code>
Categoría	<code>top_category_group</code> , <code>top_category_is_high_value</code>

El dataset de features (`05_features_rfm.parquet`) resultante tiene 96.096 filas y 51 columnas, con cero duplicados y cero nulos en las variables nuevas.

IV-B. Control de Leakage en la Construcción de Variables

La variable `top_category_is_high_value` merece mención especial: para evitar fuga del target, se definió con el cuartil superior de `avg_item_price` por categoría, y **no** con la tasa premium de la categoría (que reconstruiría parcialmente *is_premium*).

V. SELECCIÓN EXPERIMENTAL DE VARIABLES

V-A. Metodología

La selección manual fue usada únicamente como saneamiento preliminar para excluir:

- Proxies monetarias directas del target: `total_spent`, `avg_ticket`, `avg_order_price`, `avg_item_price`, `max_item_price`, `avg_freight_value`, `avg_freight_ratio`, `freight_to_item_ratio`.
- Identificadores: `customer_unique_id`, `customer_city`, `customer_zip_code_prefix`.
- Fechas crudas: `first_purchase`, `last_purchase`.

La selección final se justificó mediante **8 experimentos reproductibles**, usando un RandomForest con split temporal (train: `last_purchase < 2018-07-01` = 83.589 filas; validation: `≥ 2018-07-01` = 6.230 filas). El train fue submuestreado estratificadamente a 30.000 filas para agilizar las corridas.

Cuadro IV
RESULTADOS DE EXPERIMENTOS DE SELECCIÓN DE VARIABLES

Experimento	Método	# Feat	AUC Train	AUC Val	Gini Val
initial	initial	37	0.8172	0.7939	0.5878
missing_le_0.10	missing	37	0.8172	0.7939	0.5878
corr_le_0.90	correlation	29	0.8152	0.7923	0.5846
corr_le_0.95	correlation	32	0.8170	0.7922	0.5845
corr_le_0.85 *	correlation	28	0.8172	0.7921	0.5843
univariate_top_30 %	univariate	12	0.7941	0.7752	0.5503
univariate_top_20 %	univariate	8	0.7872	0.7686	0.5372
univariate_top_10 %	univariate	4	0.7452	0.7430	0.4860

V-B. Resultados de los Experimentos

La Tabla IV muestra los 8 esquemas evaluados, ordenados según la estrategia.

La Figura 1 ilustra la comparación entre experimentos: el número de features seleccionadas (panel izquierdo) y la evolución de AUC train vs AUC validación (panel derecho).

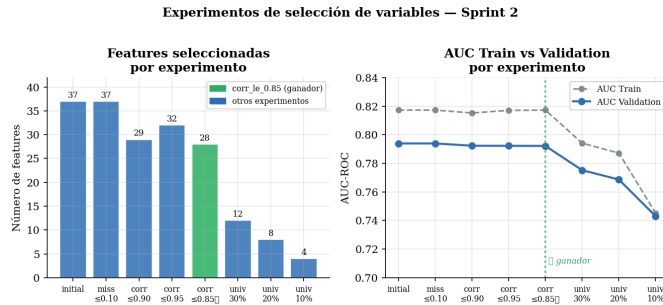


Figura 1. Comparación de experimentos de selección de variables. Panel izquierdo: número de features por experimento (verde = ganador). Panel derecho: AUC train y AUC validación; la línea discontinua vertical marca el experimento `corr_le_0.85`.

V-C. Decisión Final: `corr_le_0.85`

El experimento `corr_le_0.85` fue adoptado como set final por ofrecer el mejor equilibrio entre:

- Desempeño de validación prácticamente equivalente al set `initial` (diferencia en AUC-val de solo **0.0018**).
- Reducción de 37 a **28 features**, eliminando colinealidad interna.
- Mayor parsimonia y mejor defendibilidad metodológica frente a un jurado.

Las 28 features seleccionadas cubren dimensiones operativas (`total_orders`, `delivered_orders`), temporales (`recency_days`, `customer_lifetime_days`), de cuotas y pago, logísticas, geográficas y de categoría de producto. Ninguna de las 13 variables excluidas por leakage figura en el set final.

VI. EVALUACIÓN VS. BASELINE DEL SPRINT 1

VI-A. Configuración de la Evaluación

La evaluación se realizó con el mismo script (`src/models/evaluate_model.py`), el mismo split temporal y el mismo umbral fijo $P80 = 197.01$ que los experimentos de selección, garantizando comparabilidad directa:

- Train: 83.589 clientes (`last_purchase < 2018-07-01`)
- Validation: 6.230 clientes (`last_purchase ≥ 2018-07-01`)
- Features: 28 (set `corr_le_0.85`)

VI-B. Métricas en Validación

La Tabla V compara las métricas del Sprint 2 contra el baseline logístico del Sprint 1.

Cuadro V
COMPARACIÓN DE MÉTRICAS: BASELINE SPRINT 1 VS SPRINT 2

Métrica	Baseline S1	Sprint 2 Val	Mejora
Precision	0.3000	0.4766	+58.9 %
Recall	0.1600	0.6294	+293.4 %
F1-Score	0.2100	0.5424	+158.3 %
ROC-AUC	0.5355	0.7929	+48.1 %
Gini	0.0710	0.5858	+724.4 %

La Figura 2 ilustra visualmente el contraste entre ambas versiones del modelo.

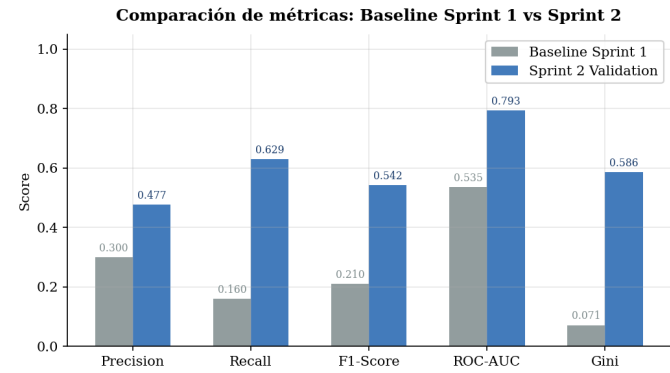


Figura 2. Comparación de métricas de clasificación entre el baseline de regresión logística del Sprint 1 y el modelo de Sprint 2 (`corr_le_0.85`, 28 features). Mejora sustancial en todas las métricas principales.

VI-C. Curva ROC y Coeficiente de Gini

La Figura 3 presenta la curva ROC del Sprint 2 junto a la curva aproximada del baseline del Sprint 1. El área bajo la curva sube de 0.5355 a **0.7929**, y el Gini de 0.071 a **0.586**, evidenciando que el enriquecimiento de variables y el control de leakage produjeron separabilidad real.

VI-D. Matriz de Confusión

Sobre el conjunto de validación (6.230 clientes), el modelo produce:

$$CM = \begin{pmatrix} TN = 3825 & FP = 983 \\ FN = 527 & TP = 895 \end{pmatrix} \quad (2)$$

El modelo identifica correctamente al **62.9 %** de los clientes premium (Recall) con una precisión del **47.7 %**. El Recall es el indicador más crítico en el contexto de negocio: detectar la mayor proporción posible del segmento de alto valor para orientar acciones de retención.

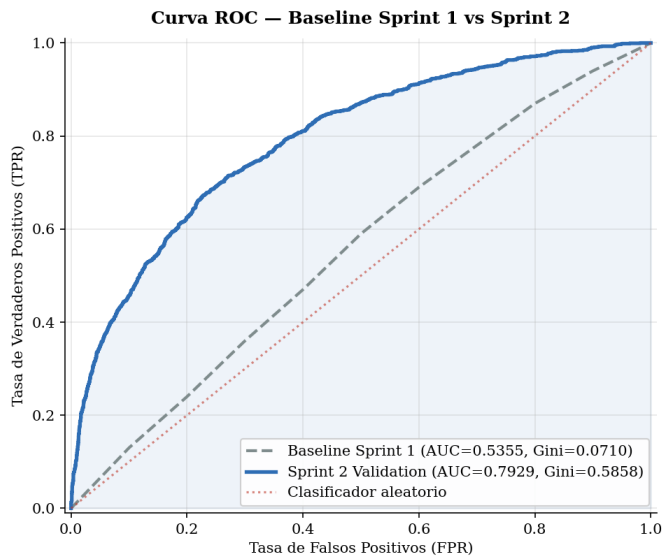


Figura 3. Curva ROC comparada entre el baseline del Sprint 1 (línea discontinua) y el modelo del Sprint 2 (línea continua). El área bajo la curva del Sprint 2 (AUC = 0.7929, Gini = 0.5858) supera ampliamente la diagonal aleatoria y el baseline previo (AUC = 0.5355).

VII. KPIs DE NEGOCIO Y DEFENSA DEL TARGET

VII-A. KPIs Observados

Los KPIs finales se calcularon sobre la master table limpia con el umbral fijo dev = 197.01. La Tabla VI resume los indicadores por segmento.

Cuadro VI
KPIs SEGMENTADOS — PREMIUM VS REGULAR

KPI	Premium	Regular
Cientes	19.220 (20.00 %)	76.876 (80.00 %)
Facturación total	BRL 8.088.732 (56.03 %)	BRL 6.348.316 (43.97 %)
Ticket promedio	BRL 402.66	BRL 92.27
Órdenes promedio	1.09	0.94
Lifetime promedio (días)	7.26	1.24
Máx. cuotas promedio	4.70	2.47
Uso de más de 1 cuota	71.55 %	42.60 %
Con entrega tardía	9.28 %	6.93 %
Recency promedio (días)	221.16	227.31

La Figura 4 ilustra la concentración de facturación y la diferencia de ticket promedio entre segmentos.

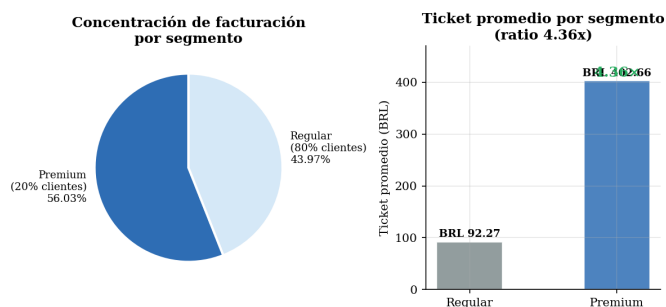


Figura 4. KPIs de negocio. Izquierda: concentración de facturación por segmento (el 20 % premium concentra el 56.03 % de la facturación). Derecha: ticket promedio por segmento (ratio 4.36x).

VII-B. Lectura de Negocio

Tres observaciones justifican la validez económica del segmento:

- El ratio **20 % / 56 %** indica que el target captura un segmento de valor real y no ruido estadístico.
- El ticket premium de BRL 402.66 es **4.36 veces** el ticket regular (BRL 92.27), coherente con la paradoja de compra única de alto valor documentada en el Sprint 1.
- El mayor lifetime (5.9x) y el mayor uso de cuotas (71.55 % vs 42.60 %) validan que las nuevas features del pipeline capturan patrones de comportamiento diferenciales reales del segmento premium.

VIII. VALIDACIÓN TEMPORAL CON HOLDOUT

El pipeline incluye un ciclo de simulación mensual sobre el periodo de holdout (agosto–octubre 2018, ventana temporal_2018q4), ejecutado con el umbral fijo calculado sobre *dev* mediante `--threshold-mode apply`. El objetivo es verificar que el umbral y las features generalizan a datos no vistos antes de pasar al Sprint 3.

La Tabla VII compara los KPIs principales entre el conjunto *dev* y el periodo de holdout.

Cuadro VII
COMPARACIÓN KPIs: DEV VS HOLDOUT (AGO–OCT 2018)

KPI	DEV	Holdout
Cientes activos	96.096	6.468
Cientes premium	19.220 (20.00 %)	1.253 (19.37 %)
Umbral aplicado	BRL 197.01 (fit)	BRL 197.01 (apply)
Ticket prem. / reg.	4.36x	4.47x

La tasa premium del holdout (19.37 %) y el ratio de ticket (4.47x) son consistentes con los valores del conjunto *dev*, lo que confirma dos propiedades del pipeline: (1) el umbral fijo no introduce sesgo temporal al aplicarse sobre una población distinta, y (2) las features del Sprint 2 capturan patrones de comportamiento estables entre periodos. La convergencia de ambas métricas valida la elección del P80 *dev* como referencia permanente para los sprints siguientes.

IX. CONCLUSIONES Y PRÓXIMOS PASOS

IX-A. Conclusiones

El Sprint 2 entregó los siguientes resultados verificables:

1. Un **pipeline modular y reproducible** en cuatro scripts independientes, con artefactos persistidos y numerados en `data/interim/` y `data/processed/`.
2. Un **target formalmente defendido**: P80 fijo sobre *dev* = BRL 197.01, con política de umbral `fit/apply` y exclusión documentada de cancelaciones.
3. **18 variables nuevas** que amplían el espacio de features más allá del RFM básico del Sprint 1.
4. Una **selección experimental** con 8 esquemas y el set óptimo de 28 features (`corr_le_0.85`), libre de las 13 variables excluidas por leakage.
5. **Mejora sustancial** en todas las métricas de clasificación: ROC-AUC 0.5355 $\rightarrow$ 0.7929 (+48.1 %), F1 0.21 $\rightarrow$ 0.54 (+158.3 %), Gini 0.071 $\rightarrow$ 0.586 (+724.4 %).

6. **KPIs económicamente sólidos:** 20 % premium, 56 % facturación, ticket 4.36× el regular.
7. **Validación con holdout:** simulación sobre agosto–octubre 2018 confirma tasa premium de 19.37 % y ratio de ticket de 4.47×, coherentes con *dev*, validando la estabilidad del umbral fijo y las features ante datos no vistos.

IX-B. Próximos Pasos — Sprint 3

Para el Sprint 3 se propone:

1. **Ventanas temporales:** separar la ventana de observación (construcción de features) de la ventana de outcome (etiquetado de *is_premium*) para una predicción verdaderamente prospectiva.
2. **Modelos de mayor capacidad:** evaluar Gradient Boosting y ensambles sobre las 28 features validadas.
3. **Análisis de valor económico:** estimar el CAC máximo recomendable por cliente premium a partir del ticket promedio y la tasa de retención estimada.
4. **Validación temporal adicional:** probar estabilidad de las features con splits de fechas distintos para confirmar robustez del pipeline.

REFERENCIAS

- [1] A. Ullah, M. I. Mohmand, H. Hussain, S. Johar, I. Khan, S. Ahmad, H. A. Mahmoud, and S. Huda, “Customer Analysis Using Machine Learning-Based Classification Algorithms for Effective Segmentation Using Recency, Frequency, Monetary, and Time,” *Sensors*, vol. 23, no. 6, p. 3180, Mar. 2023. [Online]. Available: <https://doi.org/10.3390/s23063180>
- [2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>
- [3] C. Genuer, J.-M. Poggi, and C. Tuleau-Malot, “Variable selection using random forests,” *Pattern Recognition Letters*, vol. 31, no. 14, pp. 2225–2236, Oct. 2010.